Thrift

Cost-first memory for AI agent fleets

User Guide · Apache-2.0 · internal use · early 0.0.x release

How it works — in short

The problem: every AI agent has a "memory" — a file with everything it needs to know. But on every run the agent reloads the **whole** file, including the 99% irrelevant to the current task. Run 20 agents all day and you pay again and again for the same useless context — the single biggest line on the token bill.

What Thrift does: instead of "load everything," Thrift loads **only the relevant memory** for the specific task, under a fixed token budget. The agent says "I'm fixing a bug in the proxy" — and Thrift returns only the memories related to that. Thrift filters out irrelevant memories rather than stuffing in whatever fits the budget.

And it proves it: on every load Thrift writes a receipt — what you'd have paid loading everything vs. what you actually paid. The dashboard shows the gap in real time. Not "trust me, you saved" — a measured number.

Two ways to connect: either the agent calls Thrift's memory tools directly (MCP), or you put Thrift in the middle as a proxy and change one line of config — and it trims the waste automatically, without touching the agent's code.

Install in 3 steps (MCP)

1. **Add a `.mcp.json` at your project root** with this block:

```
{
  "mcpServers": {
    "thrift": { "command": "npx", "args": ["thrift-memory"] }
  }
}
```

2. **Restart the agent** (Claude Code / Cursor / Windsurf) — it auto-detects the tools `recall`, `remember`, `search_memory`. No code rewrite.
3. **Done.** The agent now loads only the memory it needs. No manual install needed — `npx` fetches the package from npm automatically.

Option: install via npm

If you prefer to install the package instead of running through `npx`:

```
# global install
npm install -g thrift-memory

# then run directly (or use "command": "thrift-memory" in .mcp.json):
thrift-memory --store-path=~/.thrift/memories.jsonl --default-budget=2000
```

The three tools

Tool	What it does
`recall(agentId, tokenBudget, task)`	Loads only the memories relevant to the task under the token budget, and returns a receipt: `{ injectedTokens, baselineTokens, savedTokens }`. This is the one that saves the money.
`remember(scope, text, agentId?, sessionId?, tags?, pinned?)`	Cheap write path. Required: `scope` (org / agent / session) and `text`; `agentId` is only needed for agent scope. Fast and free — no mandatory LLM enrichment.
`search_memory(agentId, task?, tags?, limit?)`	Search memories without a budget — for browsing or the control panel.

Option B — Proxy (no code change)

For an agent that re-sends its whole prompt every turn, put Thrift in front as a proxy and change **only the** `base_url`. It trims and meters every request, forwards to upstream, and also handles `429` errors automatically (retry with backoff, respecting `Retry-After`).

```
npx thrift-proxy \
  --upstream=https://api.anthropic.com \
  --port=8787 \
  --budget=4000 \
  --meter-path=~/.thrift/meter.jsonl
# then point the agent's base_url to http://localhost:8787 (keep your real API key)
```

Security: by default it binds to `127.0.0.1` (local only), because the proxy forwards your real API key to upstream. Don't open it to `0.0.0.0` unless you know exactly why (via `--host` / `THRIFT_PROXY_HOST`).

The proxy's receipt is measured from the request actually sent, so it never over-reports savings.

See the savings — the dashboard

A local browser UI — no database, no build step. It reads the same files the MCP server and proxy write, and shows a fleet summary, daily flow, per-agent savings, and recent receipts.

```
npx thrift-panel serve
# open http://127.0.0.1:8585
```

> In one line: Thrift = "load only what you need — and see, in numbers, how much you saved."

Thrift — cost-first memory for AI agent fleets · Apache-2.0 · More settings (`THRIFT_STORE_PATH`, `THRIFT_DEFAULT_BUDGET`, `THRIFT_MAX_RETRIES`, and more) — in the README.